

Distributed Training & LLM Systems

Distributed Training & Large-Model Systems · Concepts

ZJUSCT

July 16, 2026

Zhejiang University Supercomputing Team

Part One · Distributed Training

- I Why We Need Parallelism & Taxonomy
- II Collective Communication
- III Data Parallelism
- IV Pipeline Parallelism (Inter-op)
- V Tensor Parallelism (Intra-op)
- VI Context Parallelism
- VII Expert Parallelism (MoE)
- VIII 6D Parallelism

Part Two · LLM Systems

- IX Transformer & Quantitative Analysis
- X LLM Inference Systems
- XI Flash Attention
- XII Frontier & Summary

Part I: Why We Need Parallelism & a Taxonomy of Parallelism

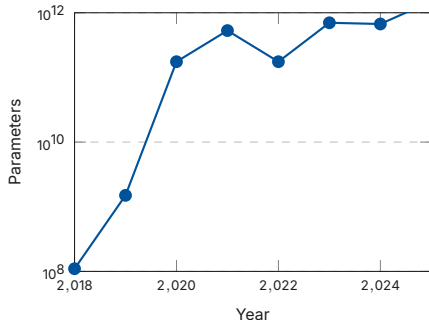


The End of Moore's Law and the Model Explosion

Key Trends

- **Moore's Law** is fading: per-GPU compute/memory growth slowing
- **Model parameters** double every 4-6 months: GPT-2 (1.5B) → GPT-3 (175B) → GPT-4 / DeepSeek-V3 (671B)
- **Memory demand** grows far faster than per-GPU memory

Conclusion: A single GPU can no longer hold a modern LLM; **distributed parallelism** is the only way out.



GPT-2 → GPT-4 / DeepSeek-V3 parameter evolution

Parallelism Paradigms: from 3D to 6D

Modern large-model training composes up to **six** parallelism dimensions. The first three are the classic **3D**; the rest extend it to **6D**.

Dimension	Splits what?
DP — Data	the input <i>batch</i> (each GPU holds a full model copy)
PP — Pipeline	<i>layers</i> across GPUs (vertical, inter-op)
TP — Tensor	<i>tensors</i> within a layer (horizontal, intra-op)
SP — Sequence	LayerNorm/Dropout activations along the <i>sequence</i>
CP — Context	KV / attention along the <i>sequence</i> (long context)
EP — Expert	<i>experts</i> across GPUs (MoE)

Remember

Inter-op splits *across* operators (= PP); *Intra-op* splits *within* operators (= DP/TP/SP/CP/EP). Classic **3D** (DP·TP·PP) + SP·CP·EP = **6D**.

Inter-op vs Intra-op Parallelism

Two Orthogonal Concepts

- **Inter-operator Parallelism:** different operators (layers) on different devices — corresponds to Pipeline
- **Intra-operator Parallelism:** split tensor dimensions within a *single operator* — corresponds to Tensor / Data Parallelism

Key observation: **Data Parallelism is Intra-op**, since it splits the batch dimension (a tensor dim), not the operator dim.

Paradigm	Inter-device communication	Typical primitive
Inter-op (Pipeline)	Point-to-point across stages	P2P send/recv
Intra-op (DP/TP)	Tensor-slice sync	Collective (allreduce, allgather)

Part II: Collective Communication

2

Point-to-Point vs Collective Communication

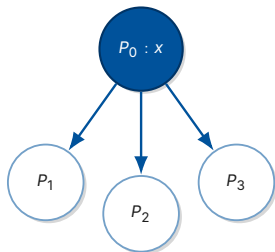
Two Communication Models

- **P2P (point-to-point)**: only two processes exchange messages directly
 - **Collective**: communication jointly involving multiple processes
-
- Collective communication cost is usually **much higher than** P2P: needs all participants to arrive
 - Collective communication can be built from multiple P2P (e.g. ring allreduce = $2N - 2$ P2P)
 - **No fault tolerance**: a single hung process blocks the whole group
 - Mainstream implementations: **NCCL** (NVIDIA), MPI, Gloo

Mnemonic

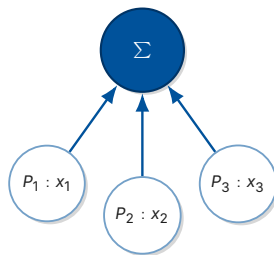
- Inter-op only needs communication between adjacent stages $\Rightarrow$ P2P
- Intra-op must synchronize the whole tensor each time $\Rightarrow$ Collective

Broadcast



single-node data x distributed to all other nodes

Reduce (-to-one)

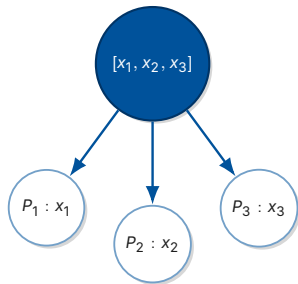


aggregate (sum/max/min) all values onto one node

Duality

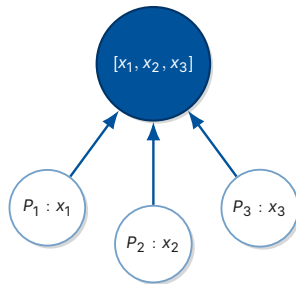
Broadcast and Reduce are often seen as inverse operations (although sum/max are mathematically not invertible).

Scatter



single-node data split into equal-size chunks sent to different nodes

Gather



gather data from multiple nodes onto a single node (inverse of scatter)

Allgather / Reduce - Scatter / Allreduce

Allgather

- each process holds a shard x_i
- afterwards every process has $[x_1, \dots, x_N]$
- = Gather + Broadcast
- **one of the most common in DP**

Reduce - Scatter

- each process has $[x_i^{(1)}, \dots, x_i^{(N)}]$
- afterwards each process holds only its own shard of $\sum_i x_i^{(k)}$
- = Reduce + Scatter
- **key operation in ZeRO**

Allreduce

- afterwards every process has $\sum_i x_i$
- = Reduce + Broadcast
- = Reduce - Scatter + Allgather
- **core op of DDP backprop**

Remember this identity

$$\text{Allreduce} = \text{Reduce-Scatter} + \text{Allgather}$$

This is the mathematical foundation of Ring - Allreduce and ZeRO optimizations.

LogP / Hockney model

The time of each point-to-point communication can be approximated as

$$T = \alpha + n\beta, \quad \beta = \frac{1}{B}$$

where α is the fixed setup overhead, B is the link bandwidth, and n is the message size in bytes.

Small messages: $\alpha \gg n\beta$

- **latency-bound**
- Goal: minimize number of communications (hops)
- Typical algorithm: **minimum spanning tree (MST)**

Large messages: $\alpha \ll n\beta$

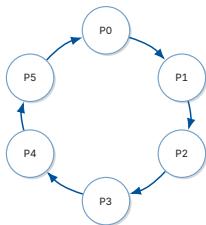
- **bandwidth-bound**
- Goal: maximize aggregate bandwidth
- Typical algorithm: **Ring Algorithm**

In practice

NCCL automatically selects based on message size and topology (NVLink / IB / PCIe):
Tree/Chain (small) → Ring (large) → bidirectional Ring / hierarchical Ring (huge).

Ring Algorithm: For Large Messages

Key idea: arrange all nodes into a logical ring and let messages flow around it.



Ring - Allreduce complexity

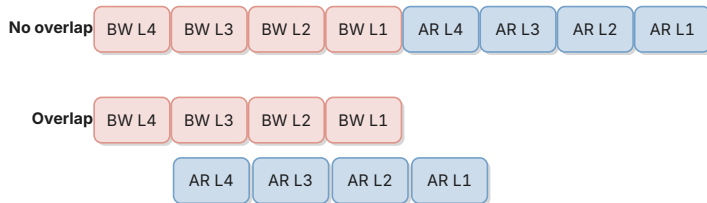
P nodes, n bytes:

$$T_{\text{ring-allreduce}} = 2(P-1)\alpha + 2 \cdot \frac{P-1}{P} \cdot n\beta$$

Bandwidth term $\rightarrow n\beta$ (independent of P) $\Rightarrow$ no matter how many GPUs, bandwidth is nearly saturated!

Communication - Computation Overlap

Observation: backprop computation and gradient allreduce can proceed in parallel – gradients computed early can be sent immediately.



Bucketing in PyTorch DDP

DDP groups gradients in parameter-registration order into several *buckets* (`bucket_cap_mb=25`); once all gradients in a bucket are ready it immediately triggers allreduce, overlapping with the backprop of subsequent layers.

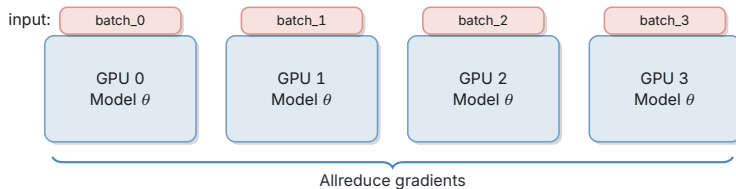
Part III: Data Parallelism

3

Data Parallelism: The Simplest Distributed Training

Assumption

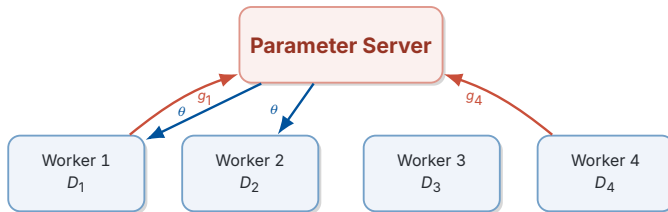
A single GPU can hold the entire model (parameters, gradients, optimizer states, activations).



One-step compute flow:

1. Each GPU does forward + backward independently, producing local gradients $g^{(k)}$
2. **Allreduce**: $\hat{g} = \frac{1}{P} \sum_k g^{(k)}$
3. Update parameters with $\hat{g}$ (identical result on every GPU)

Parameter Server: DP's Early Centralized Approach



Flow: Each worker computes $g_k = \nabla L(\theta; D_k)$ and sends it to the PS $\rightarrow$ PS aggregates and updates $\theta \leftarrow \theta - \eta \sum_k g_k \rightarrow$ broadcasts the new θ .

Two challenges

- **Communication bottleneck:** all workers must interact with the PS; PS bandwidth grows linearly with worker count
- **Single point of failure:** if the PS crashes, the whole training halts

Allreduce (Decentralized DP)

Key Idea

Each worker forms a ring directly with the others, with no centralized PS. All workers obtain the *same* global gradient via **Allreduce**.

- Introduced to ML training by Uber in *Horovod* (2017)
- NVIDIA optimized Ring Allreduce in NCCL
- PyTorch **DDP** and TensorFlow **MultiWorkerMirroredStrategy** are both based on this

	Parameter Server	Allreduce
Centralized	yes	no
Bandwidth scalability	$\propto P$	constant
Fault tolerance	single point of failure	whole group fails
Mainstream impl.	MXNet / PS-Lite	DDP / NCCL

ZeRO: Squeezing DP Memory to the Limit (Extension 1) i

Observation: standard DP stores full θ, g, m, v (Adam optimizer states) on every GPU, causing severe memory redundancy. Let the parameter count be Ψ , DP degree N , Adam constant $K \approx 12$ (fp32 master param 4 + momentum 4 + variance 4, in bytes).

Per-rank memory (bytes) decreasing by level

$$\text{Baseline DP: } \Psi(2 + 2 + K) = 16\Psi$$

$$\text{ZeRO-1 (opt): } 2\Psi + 2\Psi + \frac{K\Psi}{N}$$

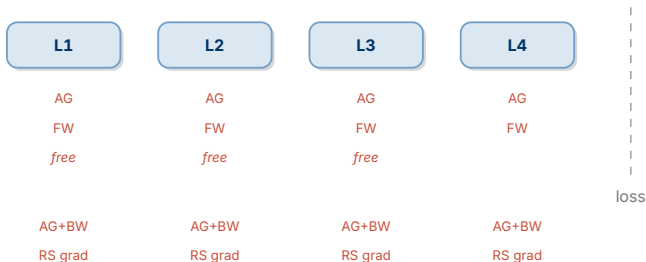
$$\text{ZeRO-2 (grad+opt): } 2\Psi + \frac{(2+K)\Psi}{N}$$

$$\text{ZeRO-3 (param+grad+opt = FSDP): } \frac{(2+2+K)\Psi}{N}$$

GPT-3 175B, $N=1024$	Baseline	ZeRO-1	ZeRO-2	ZeRO-3
Memory/GPU	≈ 2800 GB	≈ 702 GB	≈ 352 GB	≈ 2.7 GB
Comm volume	2Ψ (1 AR)	2Ψ	2Ψ	$\approx 3 \times \text{DP}$

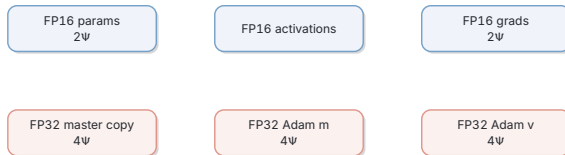
Key points and communication cost

- ZeRO-1/2 communication volume equals DDP (one reduce-scatter + one all-gather $\equiv$ one all-reduce), giving nearly *free memory savings*
- ZeRO-3 needs **all-gather parameters** before forward/backward (2Ψ) + **reduce-scatter gradients** in backward (Ψ), totaling $\approx 3\Psi$, i.e. **$3\times$ DP baseline communication**—trading bandwidth for memory
- PyTorch official implementation: **FullyShardedDataParallel (FSDP)**



- **AG**: all-gather parameter shards $\rightarrow$ full parameters $\rightarrow$ forward / backward $\rightarrow$ free immediately
- **RS**: reduce-scatter gradients $\rightarrow$ each GPU keeps only $\frac{1}{P}$ of the gradients
- Fully overlaps with backward compute (prefetch the next layer's AG)

Mixed Precision Training (Micikevicius 2018)



- **FP16 forward/backward** compute, using **loss scaling** to prevent gradient underflow
- **FP32 master copy** used for optimizer updates, avoiding accumulated rounding error
- Total per parameter = $2 + 2 + 4 + 4 + 4 = 16$ bytes

Trends

- BF16: no loss scaling needed, dynamic range = FP32
- FP8 (H100): Transformer Engine chooses precision per layer

Two hard constraints

- **Memory:** even ZeRO-3 requires a single GPU to hold *one layer's full forward + backward activations*; still strained for very large attention
- **Batch Size:** DP only splits the batch; beyond a point, growing the global batch *hurts generalization* (LR / warmup need retuning)

When the model cannot fit on a single GPU, we must partition *the model itself*:

- Inter-op $\Rightarrow$ Pipeline Parallelism (Part IV)
- Intra-op $\Rightarrow$ Tensor Parallelism (Part V)

Part IV: Pipeline Parallelism (Inter-op Parallelism)



Inter-op: Splitting the computation graph into stages

Key Idea

Partition the model's computation graph into multiple **stages**, each placed on a different device. Use *pipeline scheduling* to improve device utilization.

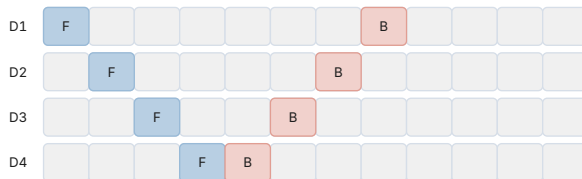
Problems with the naive approach:

- The next stage must wait for the previous stage's result → **lots of bubbles**
- Backward also requires **stashing intermediate activations**, further increasing memory pressure

Three classes of optimization

- **Device Placement**: distribute operators sensibly so stage times are balanced
- **Synchronous Schedule**: GPipe / 1F1B / Interleaved / TeraPipe / Chimera
- **Asynchronous Schedule**: AMPNet / PipeDream / PipeDream-2BW

Synchronous pipeline — naive approach



Key metrics of a pipeline

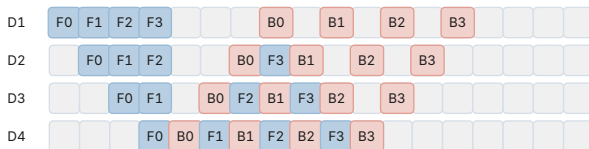
- **Bubble ratio**: fraction of idle time
- **Peak memory**: peak memory usage
- **Convergence**: whether it is exactly equivalent to single-device training

$$\nabla L_{\theta}(x) = \frac{1}{N} \sum_{i=1}^N \nabla L_{\theta}(x_i)$$


- Bubble ratio: $\frac{D-1}{D-1+N} \times 100\%$, D =stage count, N =micro-batch count
- **Cons:** must keep the activations of *all micro-batches* until the backward phase → peak memory explodes

1F1B (PipeDream Flush)

Key Idea: *Start backward as early as possible.* Once forward reaches the last stage, trigger backward immediately to free activations.



Key improvement

Peak memory drops from $O(N)$ (GPipe) to $O(D)$, independent of the number of micro-batches!

Break

Second half: Tensor Parallelism → LLM Sys

Part V: Tensor Parallelism (Intra - op Parallelism)

5

Objective: Partition the tensor dimensions of *a single operator* (matmul, conv, layernorm, etc.) so that multiple devices cooperate on the same computation.

Three Typical Scenarios

- **Element-wise:** no dependencies, arbitrary partition
- **Matrix Multiplication:** multiple partition schemes, different costs
- **Convolution:** can partition batch/channel/spatial dims

Whole-graph problem: Must also account for the *re-partition* communication cost between adjacent operators (edge cost).

Matrix Multiplication: Multiple Partition Schemes

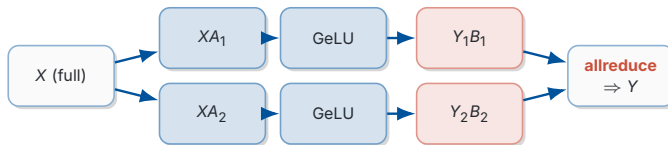
Consider $C_{m \times k} = A_{m \times n} B_{n \times k}$:

Partition scheme	Communication	Remarks
$A : S(0), B : R$	none	each device gets $C : S(0)$
$A : S(1), B : S(0)$	allreduce	each device gets a partial sum, needs aggregation
$A : S(0), B : S(1)$ (partial-tiled)	none	result $C : S(0, 1)$, "coexists across dims"
$A : S(1)$ only, $B : S(0)$	allreduce	yields partial-tiled C

Intuition for the types

- *Type-1 (Column-parallel)*: partition columns of $B \Rightarrow$ no communication
- *Type-2 (Row-parallel)*: partition rows of $B \Rightarrow$ needs **allreduce**

Megatron-LM cleverly combines these two partitions.



Column-parallel $\rightarrow$ Row-parallel: Algebraic Derivation

- **Column-parallel** $Y = XA$: partition $A = [A_1 A_2]$ along columns, then $Y_i = XA_i$, **no upfront communication**; since GeLU is element-wise, it can be done locally on shards
- **Row-parallel** $Z = YB$: partition $B = [B_1; B_2]$ along rows and $Y = [Y_1 Y_2]$, then $Z = \sum_i Y_i B_i$ is a partial sum, **needs one all-reduce**
- column-then-row means the intermediate tensor needs no all-gather; the whole MLP needs only **1 all-reduce** (1 more in backward)

Communication Volume vs. FLOPs Trade-off

Each block (QKV+O and MLP) does 2 all-reduces in forward, each moving $2bsh$ (all-reduce $\approx 2 \times$ data) $\Rightarrow$ per-block forward communication $\approx 2 \cdot 2bsh$. Compute side: with $TP=t$, per-device FLOPs drop to $1/t$, at the cost of the added constant communication above – so TP only pays off within a **high-bandwidth NVLink domain**.

- Multi-Head Attention naturally partitions along the *head* dimension
- W_Q, W_K, W_V column partition: each device computes QKV for a subset of heads
- Softmax, mask, dot-product all done within a device, **no cross-device communication**
- W_O row partition: after projecting each device's head outputs, partial sum
- one attention needs only **one allreduce**

Total Communication Volume

Each Transformer block (attention + MLP) needs **2 allreduces** (forward) + 2 (backward).

Extension: Sequence Parallelism inserts all-gather / reduce-scatter around attention/MLP to further partition the layernorm/dropout activations.

Part VI: Context Parallelism



Motivation: With long sequences (32K, 128K), the attention intermediate tensors $S, P \in \mathbb{R}^{B \times N \times S \times S}$ don't fit on a single device.

- **Sequence Parallelism (Megatron):** partition layernorm/dropout activations along the sequence dimension within a TP group
- **Ring Attention:** shard KV along the sequence onto different devices; adjacent devices form a ring and pass KV blocks round by round
- **Star Attention / Blockwise Parallel Transformer:** blockwise attention + global reduction

Ring Attention Key

Each device holds one chunk of the sequence and passes KV in a ring over P steps: $O(1)$ peak memory $\times O(P)$ communication rounds; combined with Flash Attention, it can compute ultra-long contexts losslessly.

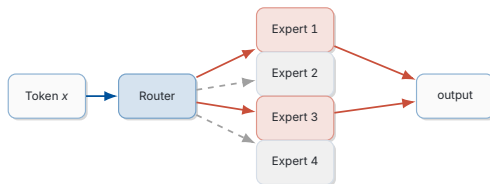
Part VII: Expert Parallelism (MoE)



Mixture of Experts (MoE)

Key Idea

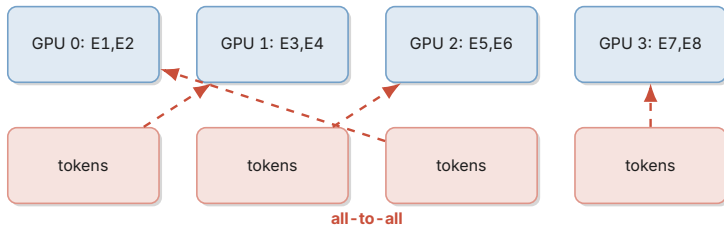
Use a **router** to select top- k experts per token, activating only those for compute.



- Parameter count grows linearly with expert count, but **compute stays nearly unchanged**
- Provides a new scale-up dimension *independent of L, H*
- DeepSeek-V3: 671B params, 37B activated; Mixtral 8×7B, Qwen1.5-MoE, etc.

Expert Parallelism in GShard - MoE

Key Idea: Place different experts on different GPUs and dispatch tokens via all-to-all.

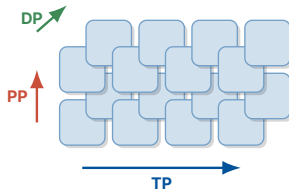


Challenges:

- Router output imbalanced → **expert load imbalance** → bubble
- Introduce *auxiliary loss* or *expert choice routing* to force balance
- Communication is all-to-all, demanding on topology

Part VIII: 6D Parallelism & Auto-parallelization





GPT-3 175B Training Config Example

- TP = 8 (NVLink within one 8-GPU A100 node)
- PP = 8 (8 machines, one stage per machine)
- DP = 60 (60 3D-parallel replicas)
- Total GPUs = $8 \times 8 \times 60 = 3840$

Auto-parallelization: How to Choose Automatically?

Problem input: Model computation graph + hardware topology (N machines $\times M$ GPUs, NVLink within each machine, IB between machines) **Problem output:** Sharding for each operator, pipeline stage partitioning, micro-batch size

Alpa Layered Optimization

1. **Inter-op stage partitioning:** DP, minimize pipeline latency
 2. **Intra-op sharding:** ILP, choose a partition strategy for operators within each stage
 3. **Device mapping:** map the logical device mesh to the physical topology
- Alpa reaches hand-tuned Megatron-level performance, fully automatic
 - OneFlow SBP, PyTorch DTensor are similar industrial implementations

Part IX: Transformer & Quantitative Analysis



$$h_t = \sum_{i=1}^t s_i x_i, \quad s_i = \text{softmax}(\text{score}(x_i, q_t))$$

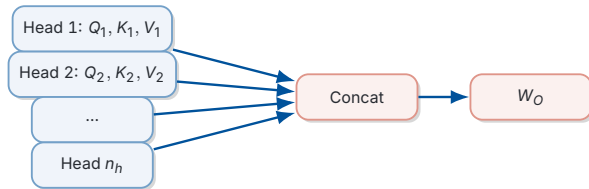
- s_i = attention score, measures importance of input x_i to output h_t
- Normalized via softmax $\rightarrow$ probability distribution
- Key difference from RNN: **parallelizable**, no dependence on previous-step output

Self - Attention Three Matrices

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Multi-Head Attention (MHA)



Parameter count: W_Q, W_K, W_V, W_O are all $H \times H$, total $4H^2$ (not $3H^2$!)

Why $4H^2$?

Each head processes H/n_h dims; after concatenating heads it is still H dims, requiring an $H \times H$ output projection W_O to map the concat result back to hidden dim.

Encoder-Decoder Block

- Multi-Head Attention
- Layer Norm + Residual
- Feed-Forward (FFN)
- Layer Norm + Residual

Masked (Causal) Attention

- Decoder cannot see future tokens during training
- Upper triangle set to $-\infty$ (equivalent to 0 after softmax)
- **Parallelizable** — key advantage over RNN
- Implemented via additive mask, no actual $-\infty$ needed

Feed-Forward Layer

$$\text{FFN}(x) = \sigma(xW_1)W_2, \quad W_1 : H \times I, \quad W_2 : I \times H$$

GPT: $I = 4H$; LLaMA-2: $I = \frac{2}{3} \cdot 4H$, with SwiGLU making total params $3HI$.

- **Pre-Norm** (LayerNorm before residual): more stable deep training
- **RMSNorm** (LLaMA): drops centering, faster
- **RoPE** (rotary position encoding): solves absolute/relative position extrapolation
- **SwiGLU**: adds gating to FFN, boosts expressiveness
- **GQA / MQA**: multiple Q heads share K/V → smaller KV cache

Total Compute

Even with these tricks, the main FLOPs still come from **Self-Attention and FFN**; other changes are just memory/bandwidth optimizations.

Parameter Count: LLaMA-2 7B Example

Structure

$$\text{Params} \approx L \cdot (4H^2 + 3HI) + V \cdot H$$

Self-Attention $4H^2$ + **FFN** $3HI$ (SwiGLU) + embedding VH .

Parameter	Symbol	LLaMA-2 7B
Layers	L	32
Hidden	H	4096
Intermediate	I	11008
Vocab	V	32000
Heads	n_h	32
Head dim	$d = H/n_h$	128

$$4H^2 = 67M/\text{layer}, 3HI = 135M/\text{layer}, \text{total } 32 \times 202M \approx 6.5B + VH = 131M \approx \mathbf{6.7B}.$$

Prerequisite: FLOPs of $M_{m \times n} \cdot M_{n \times k} \approx 2mnk$.

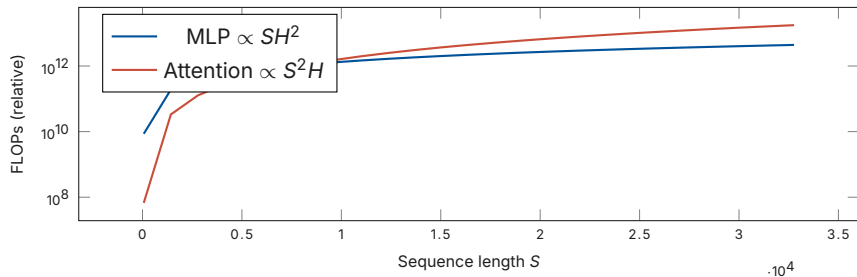
Layer type	FLOPs (per batch B , sequence S)
W_Q, W_K, W_V, W_O (4 matmuls)	$8BSH^2$
$QK^\top$	$2BS^2H$
Softmax $\times V$	$2BS^2H$
FFN (3 matmuls, SwiGLU)	$6BSHI$
Per-layer total	$8BSH^2 + 4BS^2H + 6BSHI$

Rough Formula

If $S \ll H$ (short sequence), the dominant term is $8BSH^2 + 6BSHI \approx 8BSH^2(1 + 3I/(4H))$;
If $S \gg H$ (long sequence), **attention's S^2 term** becomes the bottleneck — exactly what Flash / Ring Attention address.

Scale-Up Analysis: S vs H Tug-of-War

Observation: Attention FLOPs $\propto S^2H$, MLP $\propto SH^2$.



- Short sequence ($S < H$): **MLP is the bottleneck**
- Long sequence ($S > H$): **Attention is the bottleneck**, S^2 term explodes
- Modern LLMs commonly use Flash Attention + Sliding Window / MLA and similar optimizations

Memory Estimation: Training vs Inference State

Training - State Memory Components (Adam)

Per parameter: **16 bytes** (FP16 weights 2 + FP32 master copy 4 + gradient 2 + Adam m/v 8)

Component	Per GPU (7B)	Per GPU (70B)
Params (incl. Adam)	112 GB	1120 GB
Activations (FP16, S=2K, B=1)	8-30 GB	60-200 GB
KV cache (inference)	0.5-2 GB	5-20 GB

*A single A100 (80GB) can only train a 7B model (and barely); **ZeRO / TP / PP** is required to train 70B+.*

Gradient Checkpointing

Don't save intermediate activations; recompute forward during backward → **activation memory** $O(\sqrt{L})$, at a cost of about **25% extra compute**.

Four Scale - Up Dimensions: B/S/N/D

Dimension	Partitioning	Communication
B (batch)	Data Parallelism (DP)	allreduce gradients
N (heads)	Head partitioning (Megatron column-parallel)	No cross-GPU (each GPU handles independent heads)
D (hidden)	Tensor Parallelism	allreduce (at Megatron row-parallel)
S (sequence)	<i>Hardest</i> : inner reduction dim	involves all-to-all / ring exchange

Why is the S Dimension Hardest?

S is the **reduction dimension** in attention (QK^T reduces along S); after partitioning, communication is required to get correct results. This is the core problem addressed by Ring Attention / Sequence Parallelism.

Part X: LLM Inference Systems

10

Prefill vs Decode

- **Prefill** (0-th iteration): process *all input tokens* at once → compute-bound
- **Decode** (rest): generate only *1 new token* per step → memory-bandwidth-bound

Prefill (S=1024, one shot)

D

D

D

D

D

...(S=1 per step)

- Prefill is like the training forward pass, can saturate the GPU
- Decode does only $S = 1$ compute per step, **GPU severely under-utilized**
- KV cache is key to decode-stage performance and memory

Definition

At each decode step, save attention keys/values to avoid recomputing K/V of past tokens.

Trade space for time.

- Prefill phase *writes* KV of all tokens
- Decode phase *reads* existing KV, computes only the new token's KV and appends
- KV cache size = $2 \cdot L \cdot n_h \cdot d \cdot S \cdot B \cdot \text{sizeof}(\text{dtype})$
- LLaMA-2 7B, $S=2048$, $B=1$, FP16: ≈ 1 GB

Q: Why not cache Q?

Q is the query vector of the current new token, different each step (set by the current token), no caching value. K/V are historical vectors, reused across steps, so worth caching.

Key Observation

MLP is *token-independent*, can be concatenated arbitrarily into a large $(BS) \times H$ matrix.

Request-level (traditional) vs Iteration-level (continuous) scheduling:

- **Request-level**: a batch of requests enters the GPU together, the shortest waits for the longest to finish; new requests wait for the whole batch to end—GPU idle for long
- **Iteration-level (Orca/vLLM)**: re-form the batch after each generated token (one iteration); requests can be **added/removed at any time**, finished ones exit, new ones injected immediately

Different impacts on TTFT and TPOT

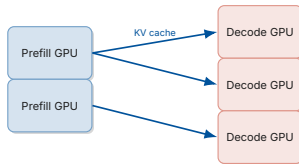
- **TTFT** (time-to-first-token) is set by **prefill queueing + prefill compute**: continuous batching lets new requests skip waiting for the whole batch, **TTFT drops significantly**; but if it competes for compute with a large decoding batch, prefill raises the TPOT of running requests
- **TPOT** (time-per-output-token) is set by the **per-iteration batch size / memory access** in the decode phase: fuller batch means higher throughput, but mixing in prefill (chunked prefill mitigates) causes periodic jitter
- Conclusion: throughput (aggregate TPOT) and latency (TTFT) often trade off, need scheduling policies (e.g., priority, chunked prefill) to balance

*Measured: vs static batching, throughput improves **up to $\sim 20\times$** (depends on load/length distribution).*

Problem with Continuous Batching

Prefill (compute-bound) and Decode (memory-bandwidth-bound) compete for resources on the same card, **TTFT jitter**.

Solution: deploy Prefill and Decode to *different GPU pools*



- Each phase scales independently
- Needs **efficient KV migration** (NVLink / RDMA)
- Examples: DistServe, Splitwise, DeepSpeed-Inference

Inspiration

Borrows the OS **virtual memory** and **paging** mechanisms.

Waste in traditional KV cache:

- **Reservation:** reserve memory for the worst case
- **Internal fragmentation:** waste from over-reservation
- **External fragmentation:** fragmentation from concurrent requests



Part XI: Flash Attention



Problems with the standard attention implementation:

- Must explicitly materialize $S = QK^T \in \mathbb{R}^{B \times N \times S \times S}$
- At $S = 32K$, a single head's S is $32^2M = 1$ GB
- Repeatedly moving the two large matrices S, P between HBM $\leftrightarrow$ SRAM incurs huge communication overhead

Core question

Can we **avoid materializing** S and directly obtain the attention output?

Answer: yes! The key is to cleverly organize the softmax + matmul computation so all intermediate results stay in SRAM.

Flash Attention benefits summary

Pros & Cons

- **Pros:** avoid materializing the bs^2n S matrix; greatly reduce HBM $\leftrightarrow$ SRAM memory movement
- **Cons:** slightly increases compute (rescale and re-reading K/V)
- *Overall performance:* training speedup 2-4 $\times$, inference speedup depends on S

Downstream impact

- Makes $S = 32K/128K$ training possible
- Can *skip* gradient checkpointing $\rightarrow$ saving about 25% compute
- Flash-Attention v2/v3 further exploit warp-level parallelism / TMA / asynchronous wmma
- Combined with Ring Attention, supports unbounded context length

Part XII: Frontier & Summary

12

Speculative Decoding: $B = 1$ scenario speedup

Key Idea: use a *small model* to guess the next k tokens, and verify with the large model in *one forward pass*.



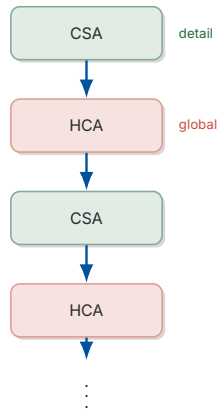
- Speedup $\propto$ average accept length (typically $2 - 4\times$)
- Variants: Medusa (multiple heads guess in parallel), Eagle (feature-based drafting)
- DeepSeek-V3: **Multi-Token Prediction (MTP)** learns multiple future tokens already at training time

DeepSeek-V4 (2026): CSA + HCA Dual-Stream Hybrid

- Scale: **1.6T** total / 49B active (MoE); native **1M-token** context
- Two attention types stacked **1:1 alternately**:
 - **CSA** (Compressed Sparse Attention): compress KV first, then top- k sparse — **detail-preserving**
 - **HCA** (Heavily Compressed Attention): more aggressive compression — **coarse global view**
- Reuses **DeepSeekMoE + MTP**; **Muon** optimizer at 1.6T scale

Idea

V3's MLA only **compresses** KV; V4 **alternates CSA + HCA** to combine **sparsity + compression** — both fine detail and a global view under 1M context.



	Within a GPU	Within a node (NVLink)	Across nodes (InfiniBand)
Bandwidth	3 - 4 TB/s HBM	900 GB/s (NVLink4)	400 Gbps (NDR)
Latency	100 ns	$\sim 1 \mu\text{s}$	$\sim 5 \mu\text{s}$
What to place	Compute + tile	Tensor Parallelism	Pipeline / Data Parallelism

- **TP** needs multiple allreduces per layer, must stay in the NVLink domain
- **PP** each stage only needs P2P, can span machines
- **DP** only one gradient allreduce per iteration, cross-machine cost is acceptable
- **DGX SuperPOD**: connects 32 nodes of 8-GPU H100 into one large mesh via NVLink Switch

Distributed training part

- **Why:** end of Moore's law + model explosion → must parallelize
- **Six parallelism dimensions:** DP / TP / PP (classic **3D**) + SP / CP (sequence) + EP (experts) = **6D**, all composable
- **Communication foundations:** MST (small messages, low latency) and Ring (large messages, high bandwidth)
- **DP family:** PS → Allreduce (DDP) → ZeRO 1/2/3 → FSDP
- **PP family:** GPipe → 1F1B → Interleaved → TeraPipe → Chimera
- **TP family:** single-operator partitioning → Megatron → GSPMD/Alpa auto-parallelism
- **Communication-computation overlap** is the common backdrop of all optimizations

LLM systems part

- **Transformer accounting:** $4H^2 + 3HI$; FLOPs $8BSH^2 + 4BS^2H + 6BSHI$
- **Scale-up dimensions:** B (DP) / N (heads) / D (Megatron) / S (Ring)
- **MoE:** adds an independent scale dimension, at the cost of all-to-all and load balancing
- **Prefill vs Decode:** fundamentally different; disaggregated deployment or merged scheduling depends on the goal
- **KV Cache:** Paged Attention $\Rightarrow 3\times$ throughput
- **Continuous Batching:** Iteration-level scheduling $\Rightarrow 20\times$ throughput
- **Flash Attention:** does not materialize S , fuses online safe softmax + tiling
- **2026 frontier:** DSA / CSA+HCA / hybrid SSM, attention diversifies from $O(n^2)$

Distributed systems + hardware awareness + algorithmic innovation = the foundation of modern LLMs

Data Parallelism / ZeRO / FSDP

- [1] S. Rajbhandari et al., "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models," *SC'20*.
- [2] A. Sergeev, M. Del Balso, "Horovod: Fast and Easy Distributed Deep Learning in TensorFlow," 2018, arXiv:1802.05799.

Pipeline Parallelism

- [3] Y. Huang et al., "GPipe: Efficient Training of Giant Neural Networks Using Pipeline Parallelism," *NeurIPS'19*.
- [4] D. Narayanan et al., "PipeDream: Generalized Pipeline Parallelism for DNN Training," *SOSP'19*.
- [5] S. Li et al., "Chimera: Efficiently Training Large-Scale Neural Networks with Bidirectional Pipelines," *SC'21*.
- [6] Z. Wang et al., "TeraPipe: Token-Level Pipeline Parallelism for Training Large-Scale Language Models," *ICML'21*.

Tensor Parallelism / Auto-Parallelism

- [7] M. Shoeybi et al., "Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism," 2020, arXiv:1909.08053.
- [8] D. Narayanan et al., "Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM," *SC'21*.

[9] L. Zheng et al., "Alpa: Automating Inter- and Intra-Operator Parallelism for Distributed Deep Learning," *OSDI'22*.

Attention / Long Context

[10] T. Dao et al., "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," *NeurIPS'22*.

[11] T. Dao, "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning," 2023, arXiv:2307.08691.

[12] H. Liu et al., "Ring Attention with Blockwise Transformers for Near-Infinite Context," 2023, arXiv:2310.01889.

Inference Systems

[13] W. Kwon et al., "Efficient Memory Management for LLM Serving with PagedAttention," *SOSP'23*.

[14] G.-I. Yu et al., "Orca: A Distributed Serving System for Transformer-Based Generative Models," *OSDI'22*.

[15] Y. Leviathan et al., "Fast Inference from Transformers via Speculative Decoding," *ICML'23*.

Communication / Collective Primitives

[16] E. Chan et al., "Collective Communication: Theory, Practice, and Experience," *Concurrency and Computation: Practice and Experience*, 2007.

Frontier Architectures (2024-2026)

- [17] DeepSeek-AI, "DeepSeek-V3 Technical Report," 2024, arXiv:2412.19437.
- [18] DeepSeek-AI, "DeepSeek-V3.2: DeepSeek Sparse Attention (DSA)," 2025.
- [19] DeepSeek-AI, "DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence," 2026, arXiv:2606.19348.
- [20] Zhipu AI, "GLM-5: from Vibe Coding to Agentic Engineering," 2026, arXiv:2602.15763.
- [21] MAMBA-3, *ICLR 2026*.
- [22] "Hybrid Architectures for Language Models: Systematic Analysis," 2025, arXiv:2510.04800.

Textbooks / Courses / Surveys

- [23] CSE234 W25 Course Notes, <https://hao-ai-lab.github.io/cse234-w25/>.
- [24] S. Raschka, "A Visual Guide to Attention Variants in Modern LLMs," 2026.

Q&A

Thank you!